Reviewer Pack — Minimal Geometric Entropy Bounds Disjointness Communication …

Entry 44082ef38eae · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 01:24:59 UTC

Minimal Geometric Entropy Bounds Disjointness Communication Complexity

Entry ID: `44082ef38eae` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-29 01:24:59 UTC

1. Conjecture

Field A (mathematical branch): Geometric Information Theory (Geometric entropy) **Field B** (complexity object): Communication Complexity (Disjointness)

Statement:

> ['For any n-ary Boolean function f: {0,...,n-1} -> {0,1}, the geometric entropy H_f of f is lower-bounded by Ω(n), where H_f is defined as the von Neumann entropy of the probability distribution Pr[f], and Pr[f] is the probability measure on {0,...,n-1} given by Prf = |f(x)|^2.', 'Equivalently, for any Boolean matrix M ∈ {0,1}^{N×N}, there exists a permutation π of [N] such that CC_R(M|_{[π(1)...π(N)]}) ≥ ⌊log_2(1 + N·ξ(M)/k)⌋ − 1, where ξ(M) is the spectral excess defined in Example 6 and k = ⌈log_2(N)⌉.', 'Furthermore, for any disjointness instance I with n variables and m clauses, if H_I ≥ Ω(n), then CC_R(I) ≥ Ω(m).']

Rationale (proposer's reasoning):

> ['Geometric entropy is a measure of the complexity of a probability distribution in terms of its information content. If the geometric entropy of a Boolean function is high, it implies that the function has a rich structure and requires more communication to describe.', 'By relating geometric entropy to communication complexity, this conjecture suggests a potential new tool for understanding the hardness of communication problems.', 'If proven true, this conjecture could provide insights into the inherent difficulty of disjointness, which is a fundamental problem in communication complexity.']

Taxonomy category: `COMM_DISJ` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `30baed1d96a2696d`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the geometric entropy of a Boolean function is at least Ω(n) with 95% confidence across all seeds, AND for each disjointness instance, the communication complexity exceeds Ω(m) with 90% confidence. The conjecture is falsified if any seed has a geometric entropy < Ω(n), or any instance has a communication complexity < Ω(m).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"geometric entropy" AND "communication complexity" AND disjointness` - `"von Neumann entropy" in "communication complexity" AND "disjointness"` - `"spectral excess" AND "Boolean matrix" AND communication complexity`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def geometric_entropy(f, n):
        Pr_f = [f(x)**2 for x in range(n)]
        H_f = -sum(p * math.log2(p) for p in Pr_f if p > 0)
        return H_f

    def spectral_excess(M):
        N = len(M)
        eigenvalues = []
        for i in range(N):
            v = [1] * N
            for _ in range(10):  # Power iteration method to approximate an eigenvector
                v = [sum(M[i][j] * v[j] for j in range(N)) for j in range(N)]
                v = [x / math.sqrt(sum(x**2 for x in v)) for x in v]
            eigenvalues.append(v)
        lambda_max = max(abs(eigenvalue[0]) for eigenvalue in eigenvalues)
        lambda_min = min(abs(eigenvalue[-1]) for eigenvalue in eigenvalues)
        return lambda_max - (N - 1) * lambda_min
```

```
    def communication_complexity(M):
        N = len(M)
        k = math.ceil(math.log2(N))
        return math.floor(math.log2(1 + N * spectral_excess(M) / k)) - 1

    n = random.randint(5, 40)
    f = lambda x: random.choice([0, 1])
    M = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]

    H_f = geometric_entropy(f, n)
    CC_M = communication_complexity(M)

    return {
        "metric_name": "Geometric Entropy",
        "metric_value": H_f,
        "instances_tested": 1,
        "conjecture_holds": H_f >= n,
        "counterexample": "" if H_f >= n else f"Geometric entropy {H_f} < {n}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / le
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"geometric_entropy_less_than_n\" first_failing_s
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7f512d72.py", line 65, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7f512d72.py", line 49, in run_trial
```

```
    CC_M = communication_complexity(M)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7f512d72.py", line 42, in communication
    return math.floor(math.log2(1 + N * spectral_excess(M) / k)) - 1
                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: math domain error
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing data, which means that the pre-registered support conditions could not be unambiguously met. | next: Re-run the test to ensure it completes without crashing and produces the required data for analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13340
2	preregistration	ollama_remote	glm4:latest	0	0	6334
3	novelty	ollama_remote	glm4:latest	0	0	4671
4	novelty	ollama_remote	glm4:latest	0	0	6104
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	45979
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7524
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10787
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8697
9	judge	ollama_remote	glm4:latest	0	0	8080

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 111517 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/44082ef38eae.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/44082ef38eae.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/44082ef38eae.tar.gz` (if generated)